

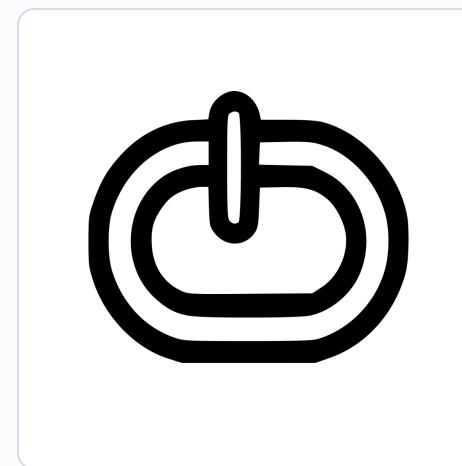
PART III DISSERTATION PRESENTATION

Fedctl: A Heterogeneous Testbed for Realistic Federated Learning

Haoran Jie

Department of Computer Science and Technology | University of Cambridge | June 2026

fedctl.cl.cam.ac.uk



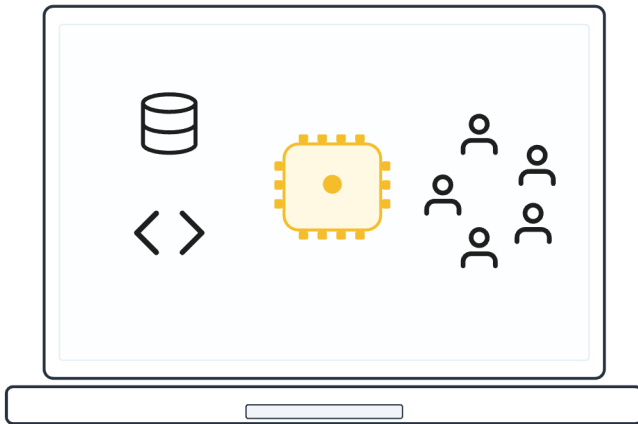
Deployment-aware evaluation of
heterogeneous FL



Heterogeneous FL Needs Deployment-Aware Evaluation

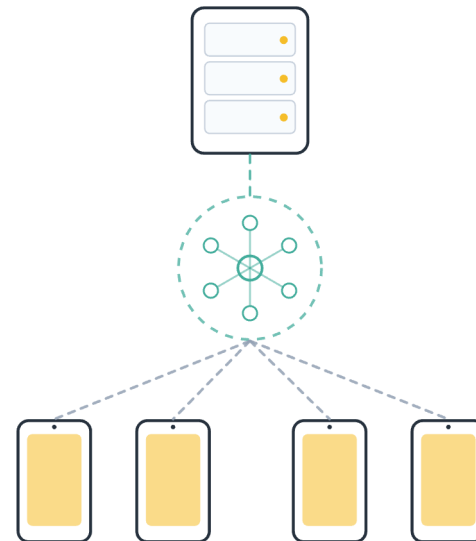
In deployed FL, a client is a device—not just a data partition. Device capability, update timing, network conditions, influence, submodel quality, and parameter coverage all shape outcomes.

Simulation



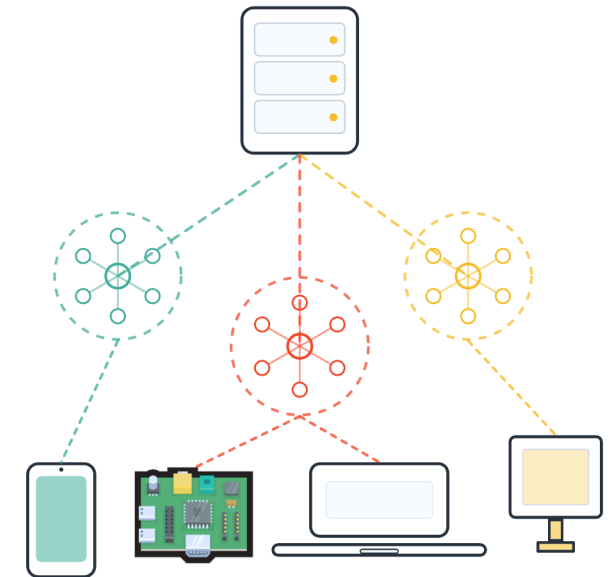
data split on one machine

Homogeneous hardware



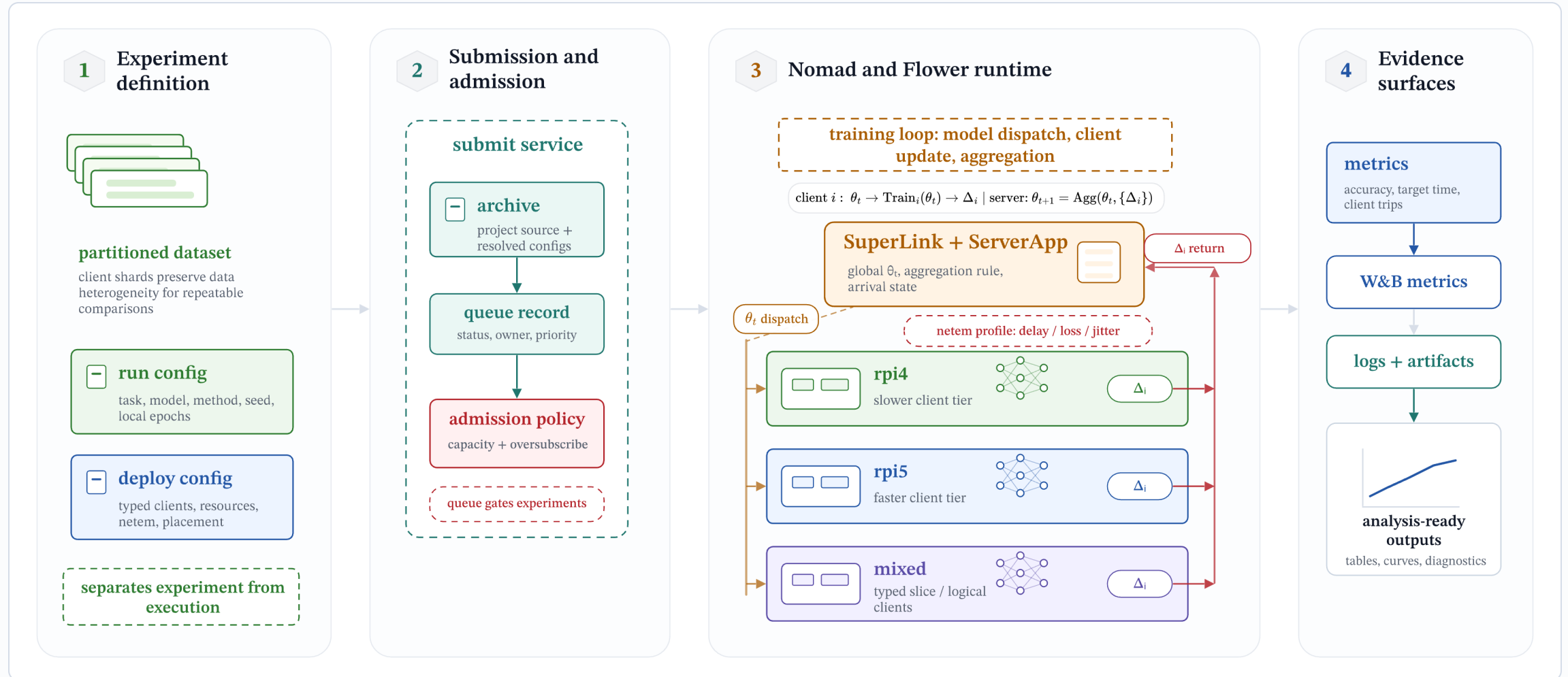
one network; equal devices

Heterogeneous hardware



mixed devices and networks

fedctl Run Lifecycle



Controlling Heterogeneity

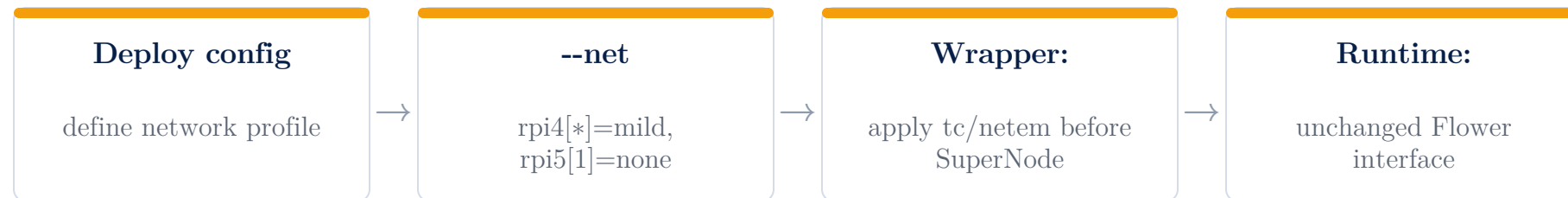
Typed client placement

Assign logical clients to device classes.

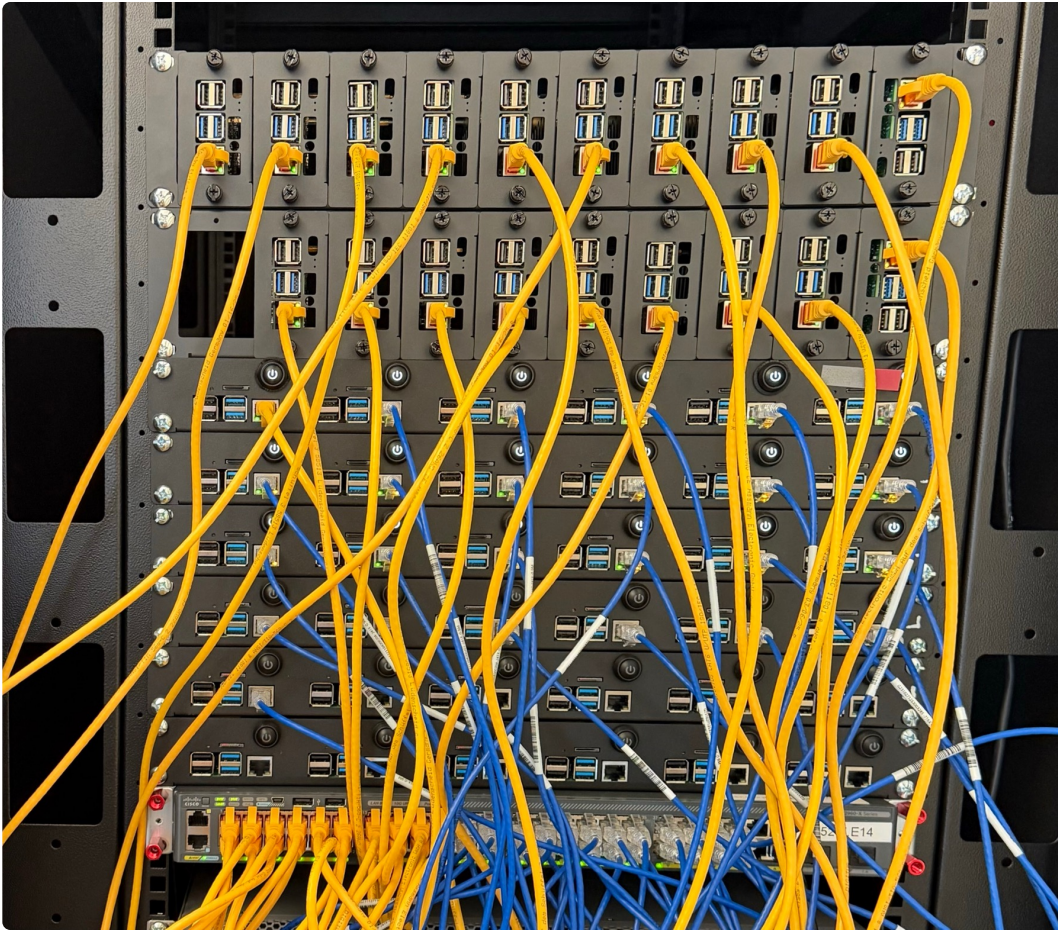


Network impairment

Apply controlled network conditions per client.



Physical Heterogeneous Cluster



50-device worker pool

30 rpi4 + 20 rpi5 for real heterogeneous execution

repeatable deployment

rerun the same workload with controlled placement and network conditions

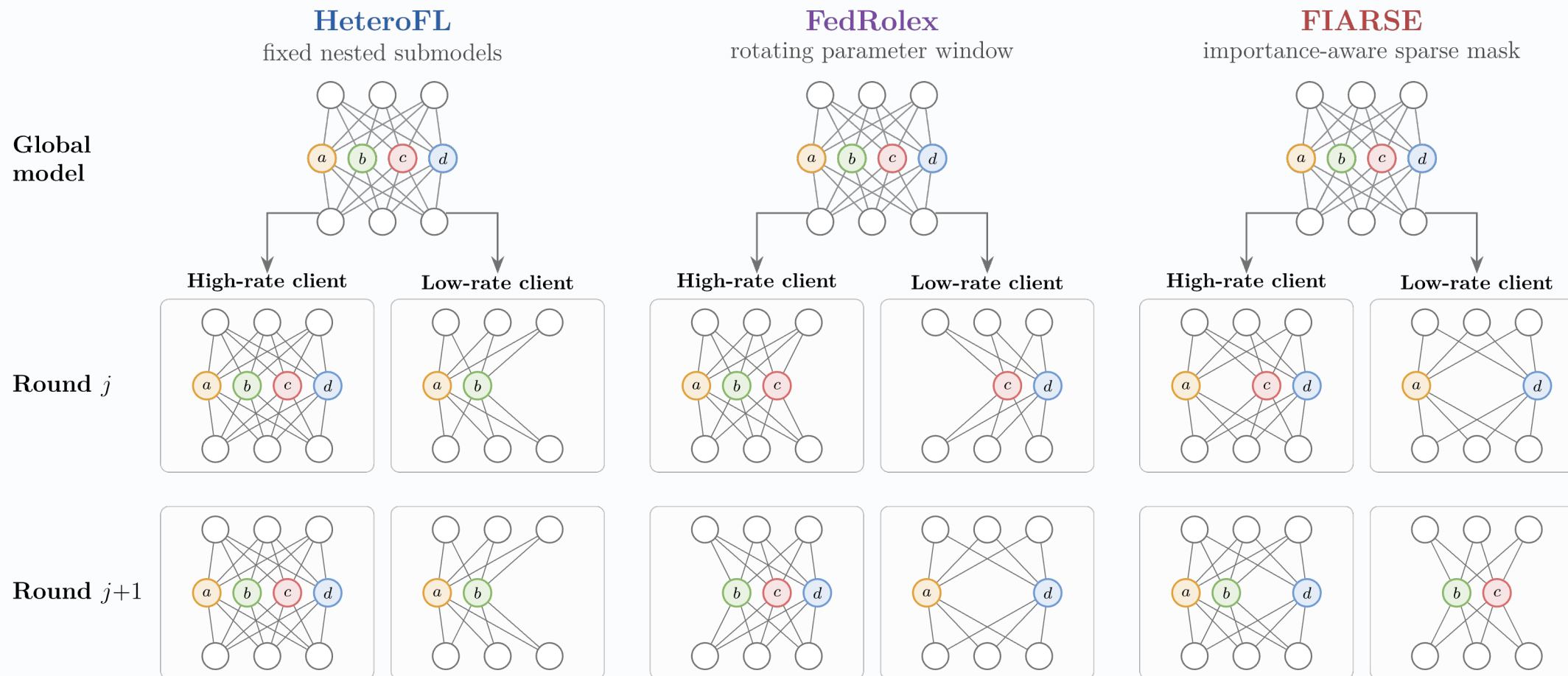
shared-lab control

queueing, resource checks, and artifacts replace manual SSH deployment

extensible devices

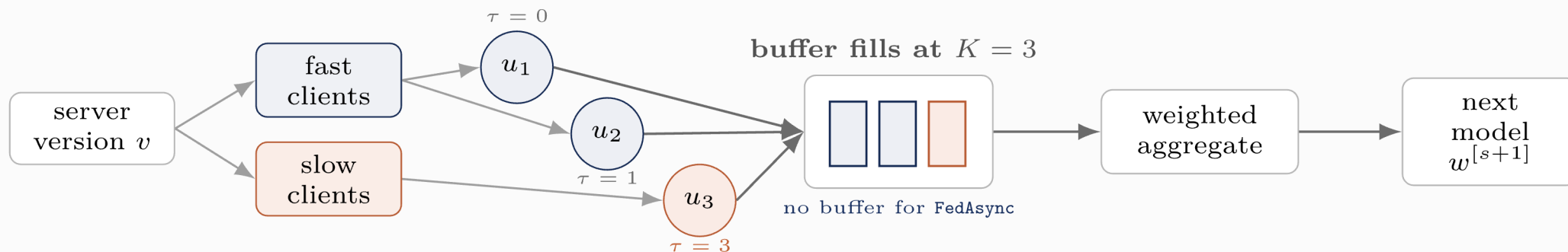
add a Jetson class and image; reuse placement and network controls

Submodel FL for Compute Heterogeneity

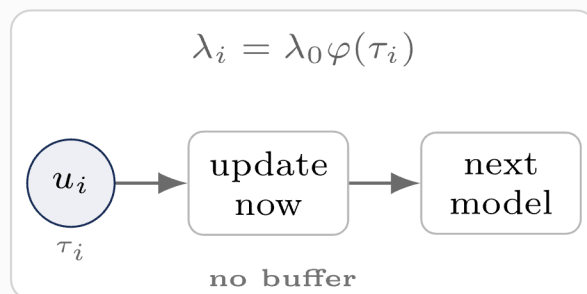


Asynchronous FL for Straggler-Induced Heterogeneity

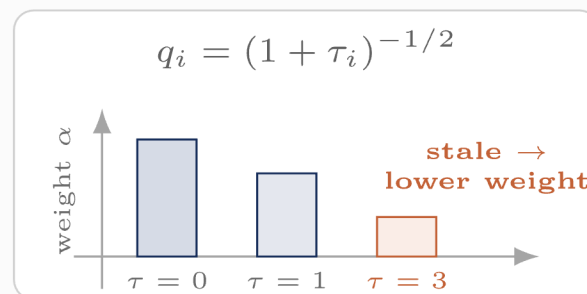
Asynchronous execution choices



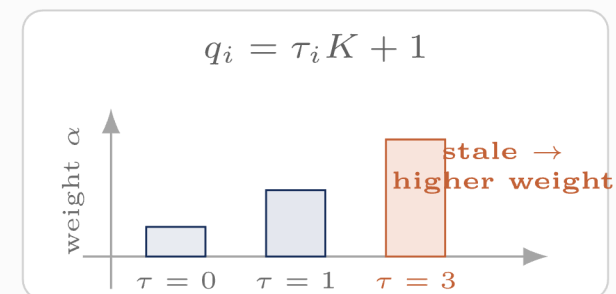
FedAsync: immediate update



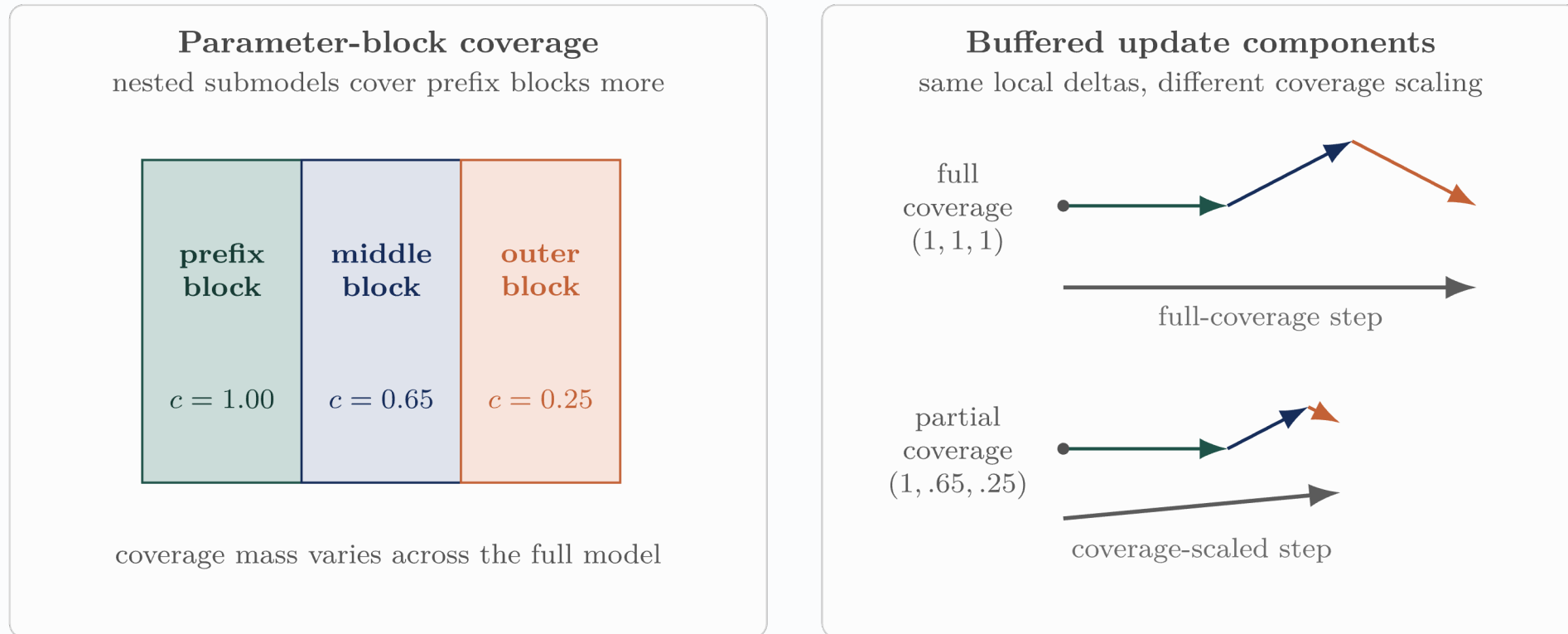
FedBuff: buffer + stale decay



FedStaleWeight: stale upweighting



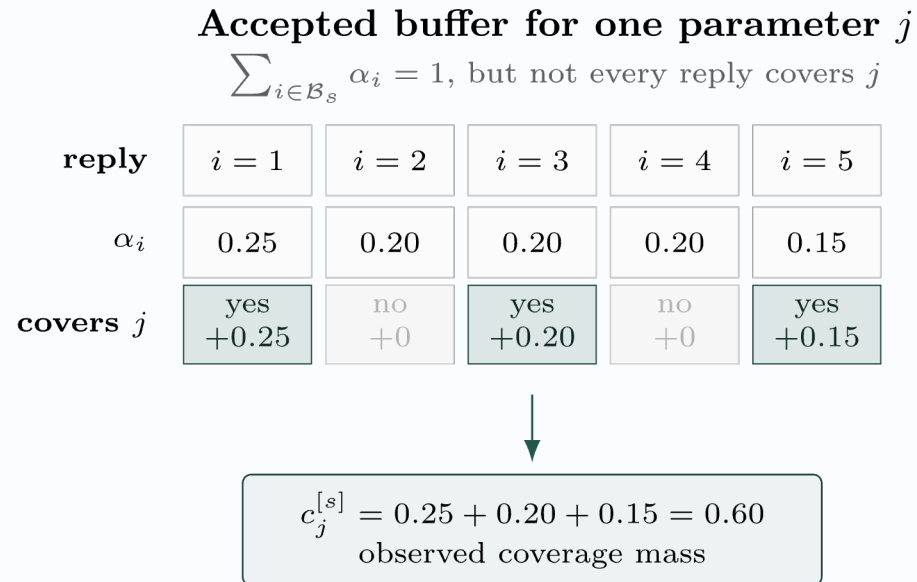
Coverage Mismatch in Asynchronous Submodel FL



Low-coverage parameter blocks can skew the buffered update direction.

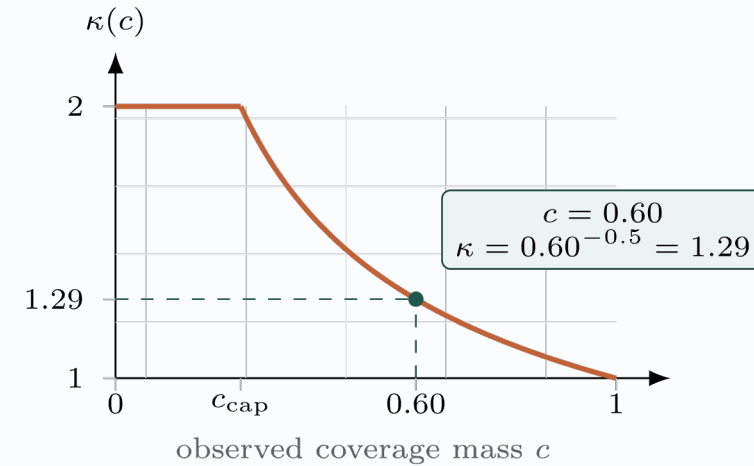
FedCover treats coverage as an aggregation variable: amplify observed under-covered deltas without imputing missing updates.

FedCover Coverage Correction



Capped inverse-coverage gain

example: $\gamma = 0.5$, $\kappa_{\max} = 2$, so $c_{\text{cap}} = \kappa_{\max}^{-1/\gamma} = 0.25$



$$\kappa(c) = \min(\kappa_{\max}, c^{-\gamma}), \text{ applied only when } c > 0$$

Server-side correction

stale-weighted buffer weight

$$\alpha_i = \frac{q_i}{\sum_{u \in \mathcal{B}_s} q_u}$$

observed coverage mass

$$c_j^{[s]} = \sum_{i \in \mathcal{B}_s} \alpha_i \mathbf{1}[j \in I_i]$$

Scale only covered deltas; do not impute missing updates.

capped coverage gain

$$\kappa_j^{[s]} = \min\left(\kappa_{\max}, (c_j^{[s]})^{-\gamma}\right)$$

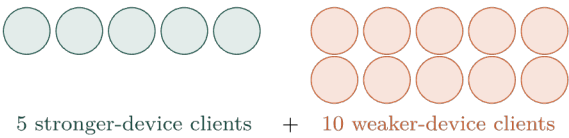
Weak-Client Inclusion Without Full-Model Cost

Reduced-rate submodels include weaker clients without the full-model runtime cost.

C1 CIFAR-10 non-IID | 5 rpi5 + 10 rpi4 | 20 rounds | compare exclusion, full inclusion, reduced-rate inclusion

Realistic non-IID deployment

15 clients hold valuable local data, but weaker-device clients are the majority.



A: Exclude
weak clients
5/15 data
fastest, but loses non-IID coverage

B: Full inclusion
all clients rate 1.0
15/15 data
direct baseline, but often infeasible

C–E: Submodel inclusion
weak clients rate 1/8
15/15 data
practical inclusion path

Measured trade-off

Case	Method	Data	rpi4 rate	Accuracy (%)	Runtime (min)
A	FedAvg	5/15	excluded	63.59 ± 2.69	38.8 ± 3.9
B	FedAvg	15/15	1.0	73.12 ± 0.95	137.5 ± 8.5
C	HeteroFL	15/15	1/8	64.30 ± 2.35	56.5 ± 15.2
D	FedRolex	15/15	1/8	62.74 ± 2.87	60.8 ± 17.6
E	FIARSE	15/15	1/8	67.83 ± 1.71	155.1 ± 14.9

HeteroFL/FedRolex keep all partitions at much lower runtime; FIARSE gives stronger reduced-rate accuracy, but no dense-kernel speedup.

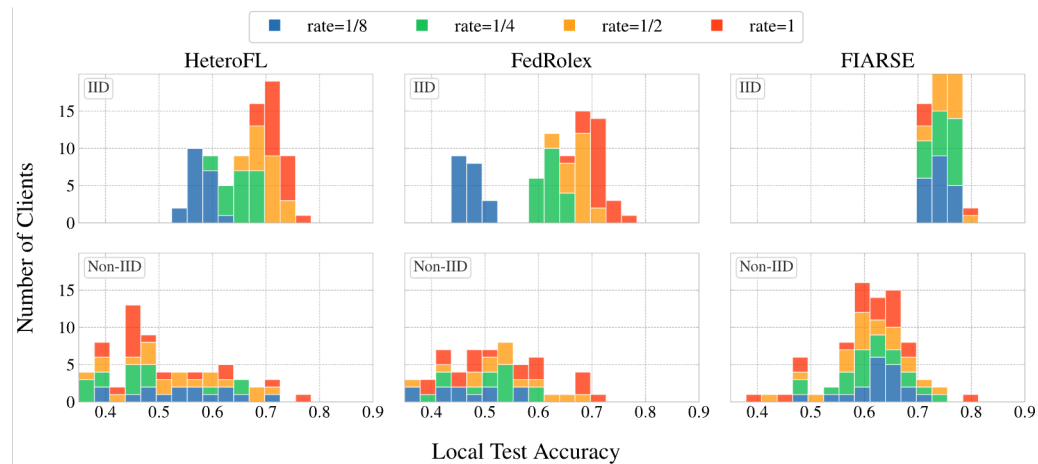
Local Submodel Performance Diagnostics

Local submodel evaluation reveals client-side quality gaps across submodel FL methods.

C3

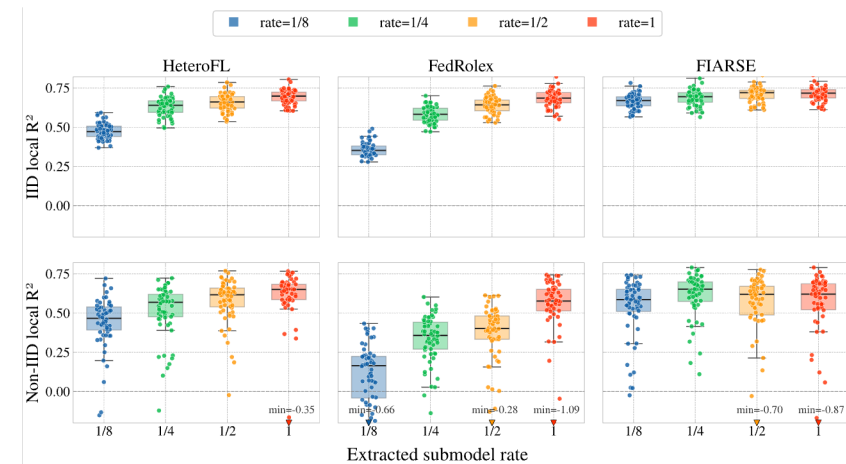
CIFAR-10 non-IID + California Housing | local evaluation of extracted rate-specific submodels

CIFAR-10 local accuracy



FIARSE gives higher and more balanced accuracy across submodel sizes.

California Housing local R^2



Smaller submodels often underperform larger ones; FIARSE narrows this gap.

Asynchronous FL Under Stragglers

Asynchronous FL helps only when client completion times are genuinely unequal.

S1 CIFAR-10 | IID/non-IID $\times$ homogeneous/mixed | target: 60% | FedAvg, FedAsync, FedBuff, FedStaleWeight

Homogeneous control

IID

FedAvg 651s best async 715s

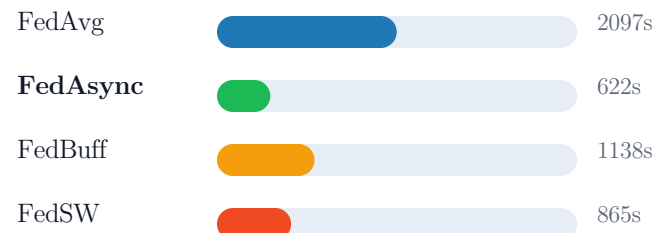
Non-IID

FedAvg 1320s best async 1486s

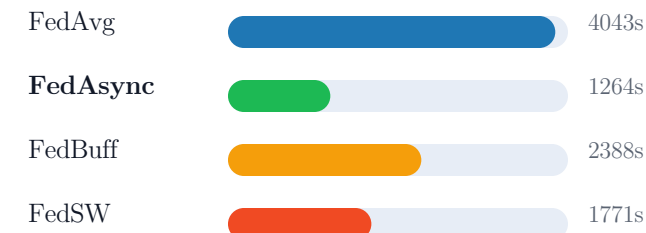
Asynchronous FL is slower in the homogeneous case.

Mixed topology: time to 60% target

IID



Non-IID



Mixed topology: asynchronous FL reduces time to target.

FedCover Improves Time to Target

FedCover improves target time; the ablation links correction strength to speedup.

A1/A2 CIFAR-10/Fashion-MNIST | IID/non-IID | async submodel FL | target time + gamma sweep

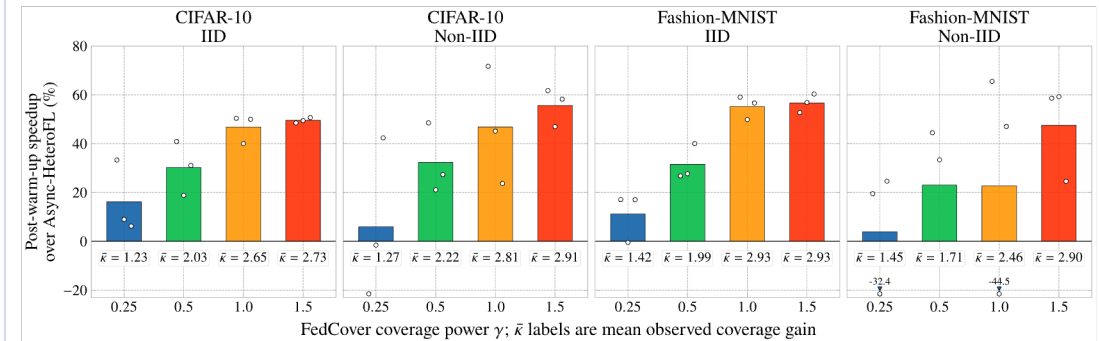
Time to task-specific target (min)

Task / data	HeteroFL	Async-HeteroFL	FedCover gamma=1.5
CIFAR-10 IID	24.2 ± 2.9	33.9 ± 16.0	13.7 ± 2.1
CIFAR-10 non-IID	51.6 ± 19.2†	38.2 ± 9.1	16.3 ± 0.3
Fashion-MNIST IID	12.9 ± 1.1	11.8 ± 1.4	6.0 ± 0.7
Fashion-MNIST non-IID	22.6 ± 2.8†	20.5 ± 6.4†	11.0 ± 2.3

Among capacity-matched methods, FedCover reaches target fastest in all four settings.

† target not reached in all seeds

FedCover Correction-Strength Ablation



Speedup generally grows with correction strength, supporting coverage correction as the mechanism.

Conclusions

Heterogeneous FL must be evaluated through both model performance and deployment behaviour.

Deployment-aware evaluation

Compute and straggler-induced heterogeneity create trade-offs between wall-clock time, participation, submodel quality, and fairness.

fedctl contribution

Repeatable Flower-to-Nomad runs preserve run/deploy configs, typed placement, network profiles, logs, metrics, and artefacts.

FedCover contribution

Async submodel FL is feasible, but stale partial replies make parameter coverage an aggregation variable; capped inverse coverage improves time to target.

Future work

- Broader workloads, devices, and runtimes.
- Representation-aware async training.
- Hardware-aware sparse submodel execution.
- Trace-driven straggler evaluation.

